

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине Построение и анализ алгоритмов
ТЕМА: «КНУТ-МОРРИС-ПРАТТ»

Студент гр. 4343

Стрижков И. А.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Цель работы

Изучить принцип работы алгоритма Кнута-Морриса-Пратта, реализовать его для решения задачи поиска вхождений заданного шаблона в текст, а также для проверки того, является ли одна строка циклическим сдвигом другой.

Задание 1

Реализуйте алгоритм КМП и с его помощью для заданных шаблона $P(|P| \leq 25000)$ и текста $T(|T| \leq 5.000.000)$ найдите все вхождения P в T .

Входные данные:

Первая строка – P

Вторая строка – T

Выходные данные:

Индексы начал вхождений P в T , разделенные запятой, если P не входит в T , то вывести -1.

Sample Input:

ab

abab

Sample Output:

0,2

Задание 2

Заданы две строки $A(|A| \leq 5.000.000)$ и $B(|B| \leq 5.000.000)$.

Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B , склеенного с префиксом B). Например defabc является циклическим сдвигом abcdef.

Входные данные:

Первая строка – A

Вторая строка – B

Выходные данные:

Если A является циклическим сдвигом B, индекс начала строки B в A, иначе вывести -1. Если возможно несколько сдвигов вывести первый индекс.

Sample Input:

defabc

abcdef

Sample Output:

3

Выполнение работы

Описание алгоритма

Алгоритм Кнута-Морриса-Пратта представляет собой метод быстрого поиска подстроки внутри строки без лишних повторных сравнений.

В основе алгоритма лежит префикс-функция, которая для каждой позиции паттерна вычисляет длину наибольшего собственного префикса, одновременно являющегося суффиксом. Эти значения позволяют при несовпадении символов не возвращаться к началу паттерна, а перепрыгивать на нужную позицию.

При поиске вхождений используется один указатель на текущую позицию в тексте и один - на текущую позицию в паттерне. На каждом шаге символы сравниваются: при совпадении оба указателя продвигаются вперёд, а при полном прохождении паттерна фиксируется найденное вхождение. При несовпадении указатель паттерна сдвигается, согласно значению, префикс-функции, что позволяет продолжить поиск с оптимальной позиции.

Для определения циклического сдвига алгоритм применяется следующим образом: поиск строки В ведётся внутри виртуально удвоенной строки А, что гарантирует наличие любого возможного сдвига.

Оценка сложности

Временная сложность алгоритма является линейной: $O(n+m)$, где n - длина текста, m - длина паттерна. Построение префикс-функции занимает $O(m)$, обход текста - $O(n)$, так как каждый символ обрабатывается не более одного раза. Для задачи циклического сдвига сложность составляет $O(2n+m)$, поскольку текст обходится дважды виртуально.

Пространственная сложность равна $O(m)$, так как требуется хранить только массив значений префикс-функции размером с паттерн (в первой задаче - размером P , во второй - размером B). Дополнительная память для хранения самих строк или результатов поиска либо константна, либо не превышает $O(n)$ в первой задаче для вывода всех позиций.

Тестирование

Обе программы были протестированы на различных входных данных. Соответственно составлены две таблицы:

Таблица 1. Тестирование задачи поиска шаблона.

Входные данные	Выходные данные
ab abab	0,2
abc abc	0
abcd abc	-1
aa aaaaa	0,1,2,3
abac ababacabac	2,6
a b	-1

Таблица 2. Тестирование задачи поиска циклического сдвига.

Входные данные	Выходные данные
defabc abcdef	3
abc abcd	-1
abc def	-1
a a	0

abcabcabc bcabcabca	7
aaaaabaa baaaaaaa	5
abcdef cdefab	2

```

PS D:\Учеба\Leti\PIAAA\lb4> cd "d:\Учеба\Leti\PIAAA\lb4\" ; if ($?) { g++ kmp.cpp -o kmp } ; if ($?) { .\kmp }
abc
abc
0
PS D:\Учеба\Leti\PIAAA\lb4> cd "d:\Учеба\Leti\PIAAA\lb4\" ; if ($?) { g++ kmp.cpp -o kmp } ; if ($?) { .\kmp }
abcd
abc
-1
PS D:\Учеба\Leti\PIAAA\lb4> cd "d:\Учеба\Leti\PIAAA\lb4\" ; if ($?) { g++ kmp.cpp -o kmp } ; if ($?) { .\kmp }
aa
aaaaa
0,1,2,3

```

Рисунок 1 – Результат работы программы kmp.cpp

```

PS D:\Учеба\Leti\PIAAA\lb4> cd "d:\Учеба\Leti\PIAAA\lb4\" ; if ($?) { g++ shift.cpp -o shift } ; if ($?) { .\shift }
defabc
abcdef
3
PS D:\Учеба\Leti\PIAAA\lb4> cd "d:\Учеба\Leti\PIAAA\lb4\" ; if ($?) { g++ shift.cpp -o shift } ; if ($?) { .\shift }
abc
abcd
-1

```

Рисунок 2 – Результат работы программы shift.cpp

Исследование

Исследуем эффективность алгоритма Кнута-Морриса-Пратта на различных объёмах входных данных. Измерения проводились на языке Python для разных размеров текста и паттерна.

Таблица 3. Исследование эффективности по времени.

Размер текста	Размер паттерна	Затраченное время (с)
1.000	1	0.000147
1.000	10	0.000154
100.000	1	0.012738
100.000	100	0.010861
100.000	10.000	0.011385
1.000.000	1	0.100852
1.000.000	1.000	0.10182
1.000.000	100.000	0.112454
5.000.000	1	0.509429
5.000.000	100	0.506219
5.000.000	10.000	0.510699
5.000.000	1.000.000	0.661234

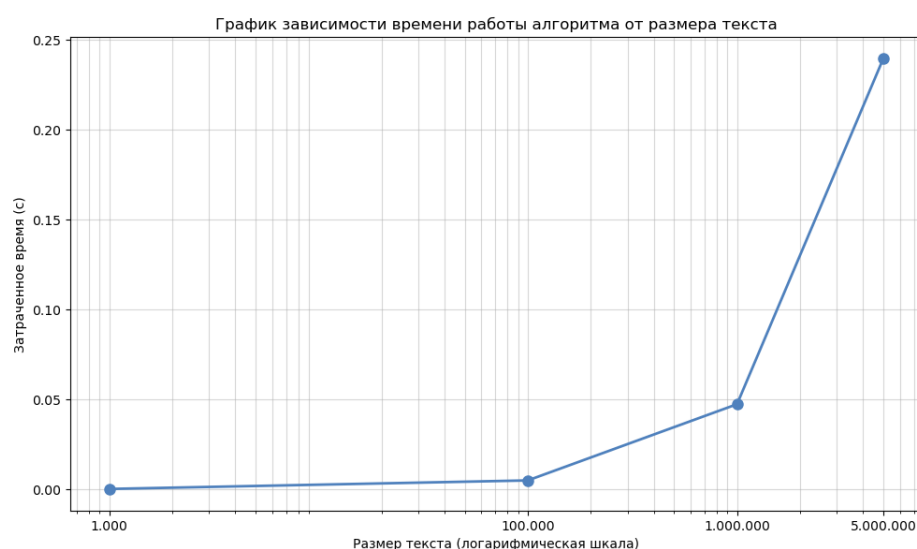


Рисунок 3 – График зависимости времени работы алгоритма от размера текста

Можно сделать следующие выводы по исследованию:

1. Время работы алгоритма прямо пропорционально размеру текста — при увеличении текста в 5000 раз время возрастает примерно в 3500 раз.
2. Время работы практически не зависит от размера паттерна, что подтверждает линейную сложность алгоритма $O(n+m)$.

Выводы

В ходе выполнения лабораторной работы был изучен алгоритм Кнута-Морриса-Пратта и принцип построения префикс-функции. Реализованы две задачи на языке Python: поиск всех вхождений заданного паттерна в текст, а также проверка того, является ли одна строка циклическим сдвигом другой.